

MPB Generator

Complete documentation — code operation, UI, user guide, troubleshooting

Summary

1. General presentation.....	5
Main features.....	5
Problem solved.....	5
2. Unity Concepts Used.....	6
Material.....	6
Shader.....	6
Shader Graph.....	6
Renderer.....	6
MaterialPropertyBlock.....	6
Material slot.....	6
3. Package Contents and Organization.....	7
4. Overall tool architecture.....	8
Architecture single-slot.....	8
Architecture multi-slot.....	8
Runtime/Editor Separation.....	9
5. Guide d'installation.....	10
Install as a local package.....	10
Simple installation in Assets.....	10
Validation after installation.....	10
6. Guide rapide : MPBGenerator single-slot.....	11
Steps.....	11
What happens when you click Generate.....	11
When to use this version.....	11
7. Guide rapide : MPBGeneratorList multi-slot.....	12
Steps.....	12
Example.....	12
Benefits.....	12
8. UI Reference: MPBGeneratorEditor.....	13
Displayed fields.....	13
Buttons.....	13
Automatic application.....	13
9. UI Reference: MPBGeneratorListEditor.....	14
General Toolbar.....	14
Box d'un slot.....	14
Dropdown des Materials.....	14
10. Runtime Code Operation.....	15
MPBGenerator: Lifecycle.....	15
Heat.....	15
GeneratePropertiesFromSourceMaterial.....	15
ApplyPropertiesToMPB.....	15
MPBGeneratorList: differences.....	16
11. Data Model: ShaderPropertyData and MPBSlot.....	17
ShaderPropertyData.....	17
MPBSlot.....	17
12. How the Shader Graph Parser Works.....	18
Entrance and exit.....	18
Principle.....	18
Why UnityEngine.Object?.....	18
A deliberate limitation.....	18

13. Recommended Shader Graph Workflow.....	19
Recommended organization.....	19
After modifying the Shader Graph.....	19
14. Shader Property Type Management.....	20
Color HDR.....	20
Range.....	20
Texture.....	20
15. Boolean and toggle handling.....	21
Current detection.....	21
Display.....	21
16. Category Management.....	22
UI side.....	22
Category Other.....	22
Order of categories.....	22
17. Material slot management.....	23
Dropdown the slot.....	23
Slot reorganization.....	23
18. Samples and example BloodHandSplatter.....	24
Recommended workflow for the sample.....	24
Smoothness control.....	24
19. Good production practices.....	25
Recommended workflow for a blood decal.....	25
20. Performance and limitations.....	26
Performance advantages.....	26
What the tool does not guarantee.....	26
Editor's Cost.....	26
22. Detailed Troubleshooting.....	27
Checklist de debug rapide.....	27
Useful logs.....	27
23. FAQ.....	28
Why not use MaterialEditor directly?.....	28
Why parse the .shadergraph?.....	28
Does the tool work with non-Shader Graph shaders?.....	28
Does MPBGeneratorList replace MPBGenerator?.....	28
Why are bools floats?.....	28
Are the values saved in the scene?.....	28
Clear MPB efface-t-il Properties ?.....	28
Can presets be created?.....	28
Why is the parser editor-only?.....	28
Should I include the hidden properties?.....	28
Appendix A — Glossary.....	29
Appendix B — Useful Snippets.....	30
Auto-assign Shader Graph.....	30
Apply per material slot.....	30
Swap de slots.....	30
Encoding-compatible buttons.....	30

1. General presentation

MPB Generator is a Unity tool designed to generate, edit, and apply Material Property Blocks from existing Materials. Its main purpose is to modify shader values per Renderer or per material slot without duplicating Materials and without having to create a different custom interface for each shader.

The tool functions like a mini Material Inspector dedicated to MaterialPropertyBlocks. It reads the properties exposed by a Material source's Shader, copies their values into serialized data, and then applies these values to a MaterialPropertyBlock. The user can then adjust the values directly in the GameObject's Inspector.

The current version is organized as a clean Unity package, with a Runtime folder, an Editor folder, shared data, documentation, and a BloodHandSplatter sample. It contains two main workflows: MPBGenerator for a single material slot, and MPBGeneratorList for multiple material slots on the same Renderer.

Main features

- Automatic extraction of shader properties from the Material of the selected slot.
- Applying values via MaterialPropertyBlock without modifying the Material asset.
- Support des types Color, Vector, Float, Range, Texture et Int.
- Displaying boolean floats as a checkbox when a Toggle attribute or a toggle/bool name is detected.
- Reading the categories of the Blackboard Shader Graph by parsing the .shadergraph file.
- Auto-detection of the source Shader Graph from the Material shader.
- Material slots dropdown based on renderer.sharedMaterials.
- Simple single-slot version and advanced multi-slot version.
- Support for a sampleBloodHandSplatterto demonstrate the workflow.

Problem solved

In Unity, directly modifying a Material can either modify a shared asset or create a new Material instance. In scenes with many objects, this can become cumbersome to maintain and lead to an unintentional proliferation of Materials.

```

Renderer
├─ sharedMaterials : Materials de base
├─ MaterialPropertyBlock global option
└─ MaterialPropertyBlock par material slot possible

Material source
└─ serves as a template to generate editable values

Shader Graph
└─ serves as the source for the custom Inspector categories
  
```

Terminology —In this document, MPB stands for MaterialPropertyBlock. MPB never refers to a Material. A MaterialPropertyBlock only contains override values applied to a Renderer.

2. Unity Concepts Used

Material

A Material is a Unity asset that references a Shader and stores the default values of that Shader's properties. In MPB Generator, the Material serves as a template. The tool reads its values, but the goal is to avoid modifying them during MPB editing.

Shader

The Shader exposes a list of properties. The tool uses Unity's reflection API to iterate through these properties: name, type, flags, attributes, range bounds, and readable description.

```
Shader shader = sourceMaterial.shader;
int count = shader.GetPropertyCount();

for (int i = 0; i < count; i++)
{
    string propertyName = shader.GetPropertyNames(i);
    ShaderPropertyType type = shader.GetPropertyType(i);
    ShaderPropertyFlags flags = shader.GetPropertyFlags(i);
}
```

Shader Graph

Shader Graph is used as the source of visual organization. Categories created in the Blackboard are visible in the Unity Material Inspector, but Unity does not provide a stable public API like `GetPropertyCategory`. Therefore, MPB Generator reads the `.shadergraph` file in the Editor to reconstruct the categories.

Renderer

The Renderer is the component that receives the `MaterialPropertyBlock`. Scripts use `RequireComponent(typeof(Renderer))` to ensure that a Renderer is present on the `GameObject`.

MaterialPropertyBlock

`MaterialPropertyBlock` stores shader property overrides for a `Renderer`. These values replace the Material's values at render time, without modifying the Material asset itself.

```
MaterialPropertyBlock mpb = new MaterialPropertyBlock();

mpb.SetFloat("_BloodIntensity", 0.8f);
mpb.SetColor("_BloodColor", Color.red);

renderer.SetPropertyBlock(mpb);
```

Material slot

A `Renderer` can have multiple Materials in `renderer.sharedMaterials`. Each entry in this array is a material slot. The multi-slot version, and now the single-slot version with dropdown, use the index of this array to target a specific Material.

```
renderer.SetPropertyBlock(mpb, 0); // Element 0
renderer.SetPropertyBlock(mpb, 1); // Element 1
```

3. Package Contents and Organization

The new package organization follows a structure compatible with modern Unity packages. It separates runtime scripts, editor-only scripts, documentation, and samples.

```
com.robinfrouin.mpb-generator/  
├── package.json  
├── README.md  
├── CHANGELOG.md  
├── LICENSE.md  
├── Runtime/  
│   ├── MPBGenerator.cs  
│   ├── MPBGeneratorList.cs  
│   └── Data/  
│       ├── ShaderPropertyData.cs  
│       └── MPBSlot.cs  
├── Editor/  
│   ├── MPBGeneratorEditor.cs  
│   ├── MPBGeneratorListEditor.cs  
│   └── ShaderGraph/  
│       └── ShaderGraphCategoryParser.cs  
├── Documentation~/  
│   ├── MPB_Generator_Documentation_EN.pdf  
│   └── MPB_Generator_Documentation_EN.docx  
├── Samples~/  
│   └── BloodHandSplatterExample/  
│       ├── Materials/  
│       │   └── BloodHandSplatter.along with  
│       ├── ShaderGraphs/  
│       │   └── BloodHandSplatterShaderGraph.shadergraph  
│       └── MakeLayer.shadersubgraph  
└── Textures/  
    ├── blood_organic_grunge_noise_1024.png  
    ├── blood_organic_grunge_noise_1024_inverted.png  
    └── Scenes/  
        └── BloodHandSplatterExample.unity
```

Path	Role
Runtime/MPBGenerator.cs	Single-slot component. Manages an MPB for a material slot selected via dropdown.
Runtime/MPBGeneratorList.cs	Multi-slot component. Manages a list of MPBSlots for multiple Materials of the same Renderer.
Runtime/Data/ShaderPropertyData.cs	Serialized data representing a generated shader property.
Runtime/Data/MPBSlot.cs	Serialized data representing a complete MPB slot in the multi-slot version.
Editor/MPBGeneratorEditor.cs	Custom Inspector for the single-slot component.
Editor/MPBGeneratorListEditor.cs	Custom Inspector de la version multi-slot.
Editor/ShaderGraph/ShaderGraphCategoryParser.cs	Parser editor-only du fichier .shadergraph.
Documentation~	Documentation ignored by the package runtime import but visible in the package.
Samples~	Sample importable from the Package Manager.

4. Overall tool architecture

The architecture is based on a simple chain: a Material provides the starting values, a Shader provides the list of properties, the Shader Graph provides the categories, and then the Editor draws a user interface that writes to a MaterialPropertyBlock.

```

Material of the selected slot
↓
Shader reflection
↓
ShaderPropertyData[]
↓
Inspector custom
↓
User modifications
↓
MaterialPropertyBlock
↓
Renderer.SetPropertyBlock(...)

```

Architecture single-slot

```

MPBGenerator
├── MaterialIndex
├── SourceMaterial
├── ShaderGraphAsset
├── Properties : List<ShaderPropertyData>
├── GeneratePropertiesFromSourceMaterial()
├── ApplyPropertiesToMPB()
├── ClearPropertyBlock()

MPBGeneratorEditor
├── dropdown Material Slot
├── auto-sync SourceMaterial depuis renderer.sharedMaterials[MaterialIndex]
├── auto-assign ShaderGraphAsset
├── boutons Generate / Apply / Clear
├── Display of properties by category

```

Architecture multi-slot

```

MPBGeneratorList
├── Slots : List<MPBSlot>
│   ├── Name
│   ├── MaterialIndex
│   ├── SourceMaterial
│   ├── ShaderGraphAsset
│   ├── IncludeHiddenProperties
│   └── Properties : List<ShaderPropertyData>
├── GenerateSlot(slot)
├── ApplySlot(slot)
├── ApplyAll()
├── ClearSlot(slot)
├── ClearAll()

MPBGeneratorListEditor
├── Add MPB Slot
├── Apply All / Clear All
├── box par slot
├── dropdown Material Slot par slot
├── Generate / Apply / Clear par slot
├── Visual reorganization of the slots

```

Runtime/Editor Separation

Runtime classes must be able to compile in a build player. Anything that depends on `UnityEditor`, `AssetDatabase`, or reading the `.shadergraph` file must remain in the Editor folder or be protected by `#if UNITY_EDITOR`.

```
#if UNITY_EDITOR
using UnityEditor;
#endif

#if UNITY_EDITOR
public UnityEngine.Object ShaderGraphAsset;
#endif
```


5. Guide d'installation

Install as a local package

1. Copy the com.robinfrouin.mpb-generator folder to a local Packages folder or to the project's Packages folder.
2. Open Unity and wait for recompilation.
3. Verify that the package appears in the Package Manager if package.json is present.
4. Import the sampleBloodHandSplatterExample from the Package Manager if needed.
5. Create or select a GameObject with a Renderer.
6. Add MPBGenerator or MPBGeneratorList.

Simple installation in Assets

7. Copy the Runtime and Editor folders into Assets/MPBGenerator.
8. Keep the Editor scripts in a folder named Editor.
9. Keep the data in Runtime/Data or in a normal folder.
10. Wait for the compilation.
11. Adding a component to a GameObject with Renderer.

Validation after installation

Test	Expected result
Add MPBGenerator	The Inspector displays a Material Slot dropdown and the Generate, Apply, Clear buttons.
Add MPBGeneratorList	L'Inspector affiche Add MPB Slot, Apply All et Clear All.
Click Generate	The list of properties populates and the categories appear if the Shader Graph is detected.
Change a value	The Renderer updates in the Scene View or Game View.
Clear MPB	The overrides are removed, without erasing the serialized data.

6. Guide rapide : MPBGenerator single-slot

MPBGenerator is the simple version. It targets only one material slot at a time, but it now uses a dropdown like the multi-slot version to prevent the user from manually assigning an inconsistent material.

Steps

12. Selecting a GameObject with a Renderer.
13. Add the MPBGenerator component.
14. Choose the Material Slot in the dropdown: Element 0, Element 1, etc.
15. Verify that SourceMaterial automatically synchronizes with the Material of the chosen slot.
16. Verify that ShaderGraphAsset is auto-assigned. If it remains empty, assign it manually.
17. Click on Refresh/Generate From Material.
18. Modify the generated properties.
19. Click on Apply MPB if necessary.
20. Click on Clear MPB to remove the slot override.

What happens when you click Generate

```
Refresh/Generate From Material
↓
AutoAssignShaderGraphAsset()
↓
GeneratePropertiesFromSourceMaterial()
↓
ShaderGraphCategoryParser.GetPropertyCategories()
↓
shader.GetPropertyCount()
↓
creating ShaderPropertyData
↓
ApplyPropertiesToMPB()
↓
renderer.SetPropertyBlock(mpb, MaterialIndex)
```

When to use this version

- Only one material to control on the renderer.
- A decal, an impact, an interactive object or a simple prop.
- A quick testing tool for a Shader Graph.
- A need for a lightweight workflow without a list of slots.

7. Guide rapide : MPBGeneratorList multi-slot

MPBGeneratorList is the advanced version. It contains a list of MPBSlots. Each slot has its own MaterialIndex, Material source, Shader Graph, and generated properties.

Steps

21. Select a GameObject with a Renderer containing one or more Materials.
22. Add MPBGeneratorList.
23. Click on Add MPB Slot.
24. In the slot box, choose a Material Slot from the dropdown.
25. Click on Generate This Slot.
26. Modify the values generated in this slot.
27. Click on Apply This Slot to apply only to this slot.
28. Repeat for the other Materials.
29. Use Apply All to apply all slots.
30. Use Clear This Slot or Clear All to remove the overrides.

Example

```
Renderer.sharedMaterials
├── Element 0 - BodyMaterial
├── Element 1 - BloodMaterial
└── Element 2 - ArmorMaterial

MPBGeneratorList.Slots
├── MPB Slot 0 → MaterialIndex 1 → BloodMaterial
└── MPB Slot 1 → MaterialIndex 2 → ArmorMaterial
```

Benefits

- A single component can manage multiple material slots.
- Each slot can have its own values and its own generation.
- The dropdown prevents index errors.
- The slots can be rearranged in the UI without changing their MaterialIndex.
- The workflow remains close to the Material Inspector, but the values remain in an MPB.

8. UI Reference: MPBGeneratorEditor

Displayed fields

Champ UI	Data source	Description
Material Slot	renderer.sharedMaterials	Dropdown displaying Element i - MaterialName. Updates MaterialIndex and SourceMaterial.
Source Material	Material you slot	Displayed as information, usually synchronized automatically.
Include Hidden Properties	bool	Includes HideInInspector properties during generation.
Shader Graph Asset	UnityEngine.Object	Asset .shadergraph automatically detected from the Material shader.

Buttons

Button	Action
Refresh/Generate From Material	Synchronizes the Shader Graph, regenerates Properties and applies the MPB.
Apply MPB	Reapplies the current values in the MaterialPropertyBlock.
Clear MPB	Removes the MaterialPropertyBlock from the targeted material slot.
Debug Categories	Displays the generated properties, categories and types in the Console.

Automatic application

At the end of OnInspectorGUI, the Editor can call ApplyPropertiesToMPB when GUI.changed is true. This provides immediate feedback in Scene View. However, it's important to avoid triggering resource-intensive operations too frequently for purely UI or selection changes.

```
if (GUI.changed)
{
    EditorUtility.SetDirty(script);
    script.ApplyPropertiesToMPB();
}
```

9. UI Reference: MPBGeneratorListEditor

General Toolbar

Button	Description
Add MPB Slot	Adds a new slot that targets the first Material of the Renderer by default.
Apply All	Applies all generated slots.
Clear All	Removes MPBs from all material slots in the Renderer.

Box d'un slot

UI element	Role
Name	Slot name is free. It is used solely for organizing the Inspector.
Move Up / Move Down	Reorganizes the display order in the list without changing MaterialIndex.
Remove	Remove the slot from the list.
Material Slot	Dropdown built from <code>renderer.sharedMaterials</code> .
Shader Graph	Asset <code>.shadergraph</code> auto-detected or manually assigned.
Include Hidden Properties	Includes hidden properties for this slot only.
Generate This Slot	Generates properties from the slot's Material.
Apply This Slot	Applies the MPB of this slot to its MaterialIndex.
Clear This Slot	Remove the MPB from this MaterialIndex.

Dropdown des Materials

```
Material[] materials = renderer.sharedMaterials;

for (int i = 0; i < materials.Length; i++)
{
    string materialName = materials[i] != null ? materials[i].name : "None";
    options[i] = $"Element {i} - {materialName}";
}

slot.MaterialIndex = EditorGUILayout.Popup(
    "Material Slot",
    slot.MaterialIndex,
    options
);
```

10. Runtime Code Operation

MPBGenerator: Lifecycle

MPBGenerator uses `ExecuteAlways` to operate in Editor mode. `OnEnable` and `OnValidate` call `Init` and then reapply the values if they exist. This ensures that the configured appearance is preserved after recompilation, selection changes, or scene reloads.

```
private void OnEnable()
{
    Heat();
    ApplyPropertiesToMPB();
}

private void OnValidate()
{
    Heat();
    ApplyPropertiesToMPB();
}
```

Heat

`Init` retrieves the `Renderer`, creates the `MaterialPropertyBlock` if necessary, and can synchronize the source `Material` according to the selected slot. In the updated single-slot version, the `Material` is primarily controlled by the Editor's dropdown.

GeneratePropertiesFromSourceMaterial

This method reads the `Shader` from the `SourceMaterial`, retrieves the `Shader Graph` categories, clears the `Properties` list, and then creates a `ShaderPropertyData` for each compatible property.

```
Shader shader = SourceMaterial.shader;
int propertyCount = shader.GetPropertyCount();

for (int i = 0; i < propertyCount; i++)
{
    string propertyName = shader.GetProperty(i);
    ShaderPropertyType propertyType = shader.GetPropertyType(i);
    ShaderPropertyFlags flags = shader.GetPropertyFlags(i);

    // HideInInspector filtering
    // Shader Graph category
    // Reading the value from SourceMaterial
    // added in Properties
}
```

ApplyPropertiesToMPB

`ApplyPropertiesToMPB` populates a `MaterialPropertyBlock` with the values stored in `Properties`. The method chooses `SetColor`, `SetVector`, `SetFloat`, `SetTexture`, or `SetInt` depending on the `ShaderPropertyType`.

```
switch (property.Type)
{
    case ShaderPropertyType.Color:
        _mpb.SetColor(property.Name, property.ColorValue);
        break;

    case ShaderPropertyType.Float:
    case ShaderPropertyType.Range:
        _mpb.SetFloat(property.Name, property.FloatValue);
        break;

    case ShaderPropertyType.Texture:
        if (property.TextureValue != null)
            _mpb.SetTexture(property.Name, property.TextureValue);
        break;
}

_renderer.SetPropertyBlock(_mpb, MaterialIndex);
```

MPBGeneratorList: differences

MPBGeneratorList does not have a single global Properties list. Each MPBSlot contains its own Material source, Shader Graph, and properties. ApplySlot creates a local MaterialPropertyBlock to prevent accidentally sharing a block between multiple material slots.

```
MaterialPropertyBlock mpb = new MaterialPropertyBlock();  
  
// Populate the MPB from slot.Properties  
  
_renderer.SetPropertyBlock(mpb, slot.MaterialIndex);
```

11. Data Model: ShaderPropertyData and MPBSlot

ShaderPropertyData

ShaderPropertyData is a serialized data structure that represents a generated shader property. It contains both the information needed by the UI and the value that can be modified by the user.

Champ	Type	Role
Name	string	Technical name used by SetFloat, SetColor, etc. Example: BloodColor.
Display Name	string	Human-readable name retrieved via shader.GetPropertyDescription.
Category	string	Display group name from Shader Graph or Other.
Type	ShaderPropertyType	Determine the UI field and the MPB method to use.
Flags	ShaderPropertyFlags	It allows, in particular, the detection of HDR or HideInInspector.
IsToggle	bool	Indicates whether a Float should be displayed as a checkbox.
FloatValue	float	Value of the Float and Range properties.
RangeMin / RangeMax	float	Slider limits for Range properties.
ColorValue	Color	Color value.
VectorValue	Vector4	Vector value.
TextureValue	Texture	Texture reference.
IntValue	int	Whole value.

MPBSlot

MPBSlot is used by MPBGeneratorList. It represents a complete generation and editing block for a Material slot of the Renderer.

Champ	Type	Role
Name	string	Name displayed in the slot box.
MaterialIndex	int	Index in renderer.sharedMaterials.
SourceMaterial	Material	Material used as a generation template.
ShaderGraphAsset	UnityEngine.Object	Asset .shadergraph used by the parser.
IncludeHiddenProperties	bool	Includes hidden properties for this slot.
Properties	List<ShaderPropertyData>	Values generated and editable for this slot.

12. How the Shader Graph Parser Works

ShaderGraphCategoryParser is an editor-only utility. It is used to retrieve the categories from the Blackboard Shader Graph, as the standard Shader API does not directly provide this information.

Entrance and exit

Element	Description
Entrance	UnityEngine.Object shaderGraphAsset, normalement un asset .shadergraph.
Exit	Dictionary<string, string> where the key is the Reference Name shader and the value is the category name.

Principle

```
.shadergraph
├── CategoryData
│   ├── m_Name = "Global"
│   └── m_ChildObjectList = property IDs
├── Vector1ShaderProperty
│   ├── m_ObjectId = "...".
│   └── m_OverrideReferenceName / m_DefaultReferenceName = "_GlobalAlpha"
├── ColorShaderProperty
└── ...
```

The parser first builds an objectId → propertyReference table, then reads each CategoryData and associates the children of the category with the category name.

```
Dictionary<string, string> objectIdToPropertyReference = new Dictionary<string, string>();
List<CategoryData> categories = new List<CategoryData>();

// 1. Read the Shader Graph properties
// 2. Read the categories
// 3. Associating child.m_Id with a property
// 4. Return propertyReference -> categoryName
```

Why UnityEngine.Object?

Shadergraph files are not always imported as TextAssets. The current version uses UnityEngine.Object to reference the asset and reads the content with File.ReadAllText from the AssetDatabase path.

A deliberate limitation

Fragility —The parser depends on the internal format of the .shadergraph files. If Unity changes this format, property generation will still work, but the categories may fall under Other.

13. Recommended Shader Graph Workflow

To achieve a clean UI in MPB Generator, the Shader Graph must be organized from the start. The Blackboard should contain readable categories and stable Reference Names.

Element	Recommendation	Example
Display Name	User-readable name	Blood Intensity
Reference Name	Stable name used in code	_BloodIntensity
Category	Short and functional name	Global, Blood, Noise, Advanced
Bool / Toggle	Toggle attribute or name containing toggle/bool	_ToggleDissolve, _IsBoolDebug

Recommended organization

```

Shader Graph Blackboard
├── Global
│   ├── _GlobalAlpha
│   ├── _GlobalNoiseStrength
│   └── _GlobalDissolveSoftness
├── Blood Layers
│   ├── _DryColor
│   ├── _MidColor
│   └── _FreshColor
├── Noise
│   ├── _NoiseScale
│   ├── _NoiseOffset
│   └── _NoiseStrength
├── Advanced
└── _DebugMode
    └── _UseVertexColor
  
```

After modifying the Shader Graph

- If a property is renamed, click Generate again.
- If a category is modified, click Generate again.
- If the Reference Name changes, the old MPB values may no longer affect the shader.
- Keep Reference Names stable once scenes use the tool.

14. Shader Property Type Management

Each ShaderPropertyType is converted into a UI field and then into a MaterialPropertyBlock method.

ShaderPropertyType	Champ UI	Stored value	MPB Method
Color	ColorField	ColorValue	SetColor
Vector	Vector4Field	VectorValue	SetVector
Float	FloatField ou Toggle	FloatValue	SetFloat
Range	Slider	FloatValue + RangeMin/RangeMax	SetFloat
Texture	ObjectField Texture	TextureValue	SetTexture
Int	IntField	IntValue	SetInt

Color HDR

If ShaderPropertyFlags.HDR is present, the Editor can open an HDR-compatible ColorField. This allows you to manage emission properties or colors with an intensity greater than 1.

Range

The Range properties are displayed in a slider with the limits retrieved via shader.GetPropertyRangeLimits. These limits are stored in ShaderPropertyData to avoid recalculating them during display.

Texture

Textures are applied only if TextureValue is not null. This prevents unintentionally overwriting a Material texture with an empty value. A future option could allow forcing SetTexture to null.

15. Boolean and toggle handling

Unity does not return a `ShaderPropertyType.Bool` for this workflow. A boolean shader is usually represented by a `Float` or an `Int` set to 0 or 1. Therefore, the tool uses an `IsToggle` field to decide whether a `Float` should be drawn as a checkbox.

Current detection

Detection must remain cautious. It relies primarily on the `Toggle` attribute, then on a fallback for names containing `toggle` or `bool`. Wildcard words like `Use` or `Enable` are not recommended for automatic detection, as they can cause false positives.

```
private static bool IsToggleProperty(Shader shader, int propertyIndex)
{
    string[] attributes = shader.GetPropertyAttributes(propertyIndex);

    foreach (string attribute in attributes)
    {
        if (attribute.Contains("Toggle"))
            return true;
    }

    string propertyName = shader.GetPropertyName(propertyIndex).ToLowerInvariant();
    string displayName = shader.GetPropertyDescription(propertyIndex).ToLowerInvariant();

    if (propertyName.Contains("toggle") ||
        propertyName.Contains("bool") ||
        displayName.Contains("toggle") ||
        displayName.Contains("bool"))
    {
        return true;
    }

    return false;
}
```

Display

```
if (property.IsToggle)
{
    bool value = property.FloatValue > 0.5f;
    value = EditorGUILayout.Toggle(label, value);
    property.FloatValue = value ? 1f : 0f;
}
else
{
    property.FloatValue = EditorGUILayout.FloatField(label, property.FloatValue);
}
```

Conseil Shader Graph — For a boolean property, add a `Toggle` attribute when possible. This is more reliable than name-based detection.

16. Category Management

A property's category is stored in `ShaderPropertyData.Category`. The runtime does not decide the category: it is calculated at generation time, then the Editor simply groups properties with the same category name.

```
string category = "Other";

if (shaderGraphCategories.TryGetValue(propertyName, out string parsedCategory))
{
    category = parsedCategory;
}

ShaderPropertyData data = new ShaderPropertyData
{
    Name = propertyName,
    Category = category,
    Type = propertyType
};
```

UI side

The Editor builds a list of unique categories by browsing Properties, then draws a section for each category.

```
List<string> categories = GetCategories(script);

foreach (string category in categories)
{
    DrawCategory(category, script);
}
```

Category Other

Other is the fallback. A property arrives in Other if the `ShaderGraphAsset` is null, if the file is not a `.shadergraph`, if the parser fails, or if the property is not attached to any recognized category.

Order of categories

The current order follows the first appearance of categories in the list of generated properties. A future version could retrieve the exact order of the `CategoryData` or apply a configured priority.

17. Material slot management

The MaterialPropertyBlock applies to the Renderer, not the Material. This is why MaterialIndex is necessary whenever a Renderer has multiple Materials, or when the same Material is used in multiple slots.

Cas	SourceMaterial is sufficient?	MaterialIndex required?
Only one material on the renderer	Yes, generally	Possible but not essential
Several different materials	Partially	Yes, to target the right slot
Same material used in multiple slots	Non	Yes, absolutely

Dropdown the slot

The dropdown reads renderer.sharedMaterials and displays Element i - MaterialName. It then synchronizes MaterialIndex and SourceMaterial.

Slot reorganization

In MPBGeneratorList, the reorder buttons do not change MaterialIndex. They only change the display order of the Slots list to make the Inspector more readable.

```

if (GUILayout.Button("\u25B2", GUILayout.Width(25)))
{
    SwapSlots(script, slotListIndex, slotListIndex - 1);
}

if (GUILayout.Button("\u25BC", GUILayout.Width(25)))
{
    SwapSlots(script, slotListIndex, slotListIndex + 1);
}

```

18. Samples and example BloodHandSplatter

The Samples~ folder contains a BloodHandSplatter example. It serves to demonstrate the complete workflow: organized Shader Graph, Material source, textures, subgraph, and demo scene.

Asset	Role
BloodHandSplatter.along with	Material source used by the generator.
BloodHandSplatterShaderGraph.shadergraph	Main Shader Graph with exposed properties and categories.
MakeLayer.shadersubgraph	Subgraph of blood layer composition.
blood_organic_grunge_noise_1024.png	Organic noise texture.
blood_organic_grunge_noise_1024_inverted.png	Inverted variant of the noise texture.
BloodHandSplatterExample.unity	Demonstration scene.

Recommended workflow for the sample

- 31. Import the sample from the Package Manager.
- 32. Open BloodHandSplatterExample.unity.
- 33. Select a demo object with Renderer.
- 34. Add MPBGenerator or MPBGeneratorList.
- 35. Choose the material slotBloodHandSplatter.
- 36. Click Generate.
- 37. Modify the values of color, alpha, noise, smoothness or offsets.
- 38. Verify that the object reacts without modificationBloodHandSplatter.along with.

Smoothness control

For a blood shader, a simple pattern is to multiply the smoothness channel by an MPB-controllable float, then saturate the result.

```
SmoothnessMapChannel
→ Multiply(SmoothnessStrength)
→ Saturate
→ Smoothness
```

19. Good production practices

- Do not modify the Material asset from the custom MPB Inspector.
- Always regenerate properties after a Shader Graph change.
- Use stable Reference Names.
- Avoid renaming properties when scenes are already using the tool.
- Leave Include Hidden Properties disabled except for specific debugging purposes.
- Use MPBGeneratorList when a Renderer has multiple Materials.
- Test Clear MPB to verify that no overrides remain.
- Keep a minimal sample that demonstrates the complete workflow.
- Document the tested Unity versions in README.md.

Recommended workflow for a blood decal

39. Create the Shader Graph properties: colors, alpha, noise, smoothness, offsets.
40. Organize the Blackboard into categories: Global, Blood Layers, Noise, Advanced.
41. Create a clean Material source with default values.
42. Add MPBGenerator to the decal if only one material slot is used.
43. Add MPBGeneratorList if multiple materials need to be driven.
44. Generate the properties.
45. Adjust the values per instance.
46. Test in Play Mode that the values remain independent between objects.

20. Performance and limitations

Performance advantages

The advantage of MPB is that it avoids duplicating materials for simple variations per object. The same material can be shared by multiple renderers, while each renderer has different values via its MPB.

What the tool does not guarantee

- It does not automatically guarantee GPU instancing.
- It does not make a property active if it is not used in the Shader Graph.
- It does not circumvent the limitations of Shader Graph.
- It does not guarantee the stability of the .shadergraph format between Unity versions.
- It does not replace the complex internal drawers of the Material Inspector Unity.

Editor's Cost

Parsing and generation primarily occur when the user clicks Generate. The MPB application is lightweight. If a scene contains many slots and textures, it may be advantageous to apply only the modified slot rather than ApplyAll with each GUI change.

22. Detailed Troubleshooting

Issue	Cause probable	Solution
All properties are in Other	ShaderGraphAsset null or parser unable to find categories	Check the Shader Graph field, click Generate, and verify the .shadergraph format.
Shader Graph not automatically assigned	AssetDatabase.GetAssetPath(shader) does not return a .shadergraph	Manually assign the .shadergraph or adapt AutoAssignShaderGraphAsset.
Erreur duplicate ShaderGraphCategoryParser	The class exists in several files	Keep only one class in Editor/ShaderGraph/ShaderGraphCategoryParser.cs.
The toggles remain in float	No toggle attribute and no name without toggle/bool	Add a Toggle attribute or rename the property.
The MPB doesn't change anything.	Incorrect material slot, unused property, or incorrect Reference Name	Check MaterialIndex, Reference Name and Shader Graph branching.
Le material asset change	The Material is modified directly or sharedMaterial is assigned to the wrong place	Do not edit the Material Inspector; check the Init logic.
UnityEditor error during build	A runtime script references UnityEditor outside of #if	Move to Editor or protect with #if UNITY_EDITOR.
Dropdown material vide	Renderer sans Materials	Assign at least one Material to the Renderer.
Arrows do not compile	Incompatible source encoding	Use \u25B2 and \u25BC instead of the copied characters.
Texture does not apply	TextureValue null or shader does not use the property	Assign a texture and check the Reference Name.

Checklist de debug rapide

47. Look at the Unity Console and correct the first error.
48. Verify that the Editor scripts are indeed in an Editor folder.
49. Verify that there is only one ShaderGraphCategoryParser.
50. Verify that the Renderer has at least one Material.
51. Verify that MaterialIndex targets the correct Element.
52. Logger AssetDatabase.GetAssetPath(SourceMaterial.shader).
53. Verify that the path ends with .shadergraph.
54. Click Generate after any changes to the Shader Graph.
55. Verify that Properties.Count is greater than 0.
56. Verify that property.Name matches the Reference Name in Shader Graph.

Useful logs

```

Debug.Log($"Shader = {shader.name}");
Debug.Log($"Shader Path = {shaderPath}");
Debug.Log($"Properties count = {script.Properties.Count}");
Debug.Log($"Property: {property.Name} | Category: {property.Category}");

```

23. FAQ

Why not use MaterialEditor directly?

MaterialEditor draws the Material Inspector and modifies the Material asset. MPB Generator must modify serialized values in a component and then apply them to a MaterialPropertyBlock.

Why parse the .shadergraph?

Because the standard Shader API does not directly provide the categories of the Blackboard Shader Graph.

Does the tool work with non-Shader Graph shaders?

Yes, to generate and apply the properties. The categories will simply fall under Other if no .shadergraph is available.

Does MPBGeneratorList replace MPBGenerator?

No. MPBGenerator is simpler for a single slot. MPBGeneratorList is useful for multiple material slots.

Why are bools floats?

This is common practice in Unity shaders. A toggle is often stored as 0 or 1.

Are the values saved in the scene?

Yes. Properties and Slots are serialized in the MonoBehaviour component.

Clear MPB efface-t-il Properties ?

No. Clear removes the Renderer override but retains the serialized data.

Can presets be created?

Yes, a future extension could use ScriptableObjects to save and reapply groups of values.

Why is the parser editor-only?

Because it uses AssetDatabase and reads the .shadergraph file, which does not exist in a build player.

Should I include the hidden properties?

Only for debugging or very specific cases. By default, it's best to hide them.

Appendix A — Glossary

Term	Definition
MPB	MaterialPropertyBlock, shader value block applied to a Renderer.
Renderer	A Unity component that draws an object and receives the Materials.
Material slot	Index of a Material in <code>renderer.sharedMaterials</code> .
Shader Graph Asset	The <code>.shadergraph</code> source file was used to retrieve the categories.
Reference Name	Technical name for shader property used in code.
Display Name	Name displayed to the user in the Inspector.
Category	Display group from the Blackboard Shader Graph.
Hidden Property	Shader property marked <code>HideInInspector</code> .
Toggle	Boolean shader usually stored as float 0/1.
Sample	Example that can be imported from <code>Samples~</code> into the Package Manager.

Appendix B — Useful Snippets

Auto-assign Shader Graph

```
Shader shader = sourceMaterial.shader;
string shaderPath = AssetDatabase.GetAssetPath(shader);

if (!string.IsNullOrEmpty(shaderPath) && shaderPath.EndsWith(".shadergraph"))
{
    Object shaderGraphAsset = AssetDatabase.LoadMainAssetAtPath(shaderPath);
}
```

Apply per material slot

```
MaterialPropertyBlock mpb = new MaterialPropertyBlock();

// Fill in the MPB

renderer.SetPropertyBlock(mpb, materialIndex);
```

Swap de slots

```
private static void SwapSlots(MPBGeneratorList script, int a, int b)
{
    Undo.RecordObject(script, "Reorder MPB Slots");

    MPBSlot temp = script.Slots[a];
    script.Slots[a] = script.Slots[b];
    script.Slots[b] = temp;

    EditorUtility.SetDirty(script);
}
```

Encoding-compatible buttons

```
GUILayout.Button("\u25B2", GUILayout.Width(25)); // triangle haut
GUILayout.Button("\u25BC", GUILayout.Width(25)); // triangle bas
```